

Cosmos — Source Code Structure Review

Small, targeted fixes for readability, correctness and performance

Reviewed by Claude Sonnet 5

August 20, 2026

Contents

Scope and method	2
1. Correctness bugs worth fixing first	3
1.1 <code>Field.cf</code> setter crashes with the wrong exception type	3
1.2 <code>CosmosModel.create_ale</code> compares the wrong attribute	3
1.3 <code>set_config</code> silently accepts typos and never interpolates the key	3
1.4 Volume ADR's nonlinearity hook breaks for a single-species system	3
2. Memory and performance	4
2.1 The stabilised boundary-ADR model rebuilds its whole linear system every time step	4
2.2 <code>implicit_solve_step_gauss</code> copies every PDE's full vector on every sub-iteration	4
2.3 <code>_extend_displacement_to_bulk</code> reassembles the same matrix twice per step	4
2.4 Dead, commented-out code left in place instead of relying on git history	5
3. Class declaration, construction, and object design	6
3.1 Redundant re-assignment right after <code>super().__init__()</code>	6
3.2 <code>Optional[...]</code> typing is inconsistent between <code>base.py</code> and its subclasses	6
3.3 Managers are inconsistent about whether they hold a back-reference to <code>model</code>	6
3.4 Config kwargs are threaded through four managers with no validation	7
4. Dead and unused code	8
5. File and folder organisation	9
5.1 Two unrelated classes share the exact same name	9
5.2 Inconsistent nesting depth inside <code>pde/</code>	9
5.3 <code>utils/dendritic_spine_geom.py</code> is domain-specific, not a generic utility	9
5.4 Subpackage <code>__init__.py</code> files are all empty	9
6. Logging, error handling, and tooling conventions	11
6.1 A library shouldn't call <code>logging.basicConfig()</code> on import	11
6.2 <code>print()</code> instead of the already-imported logger, inconsistently	11
6.3 Swallowed exception loses its original type and traceback	11
6.4 One stray formatting slip	12
6.5 No lint/format/type-check tooling configured	12
7. Summary table	13

Scope and method

This is a walkthrough of `src/cosmos` (config, core, io, pde, utils — 4,265 lines across 22 files) looking for small, local improvements: correctness bugs, memory/ performance issues, class-design inconsistencies, dead code, and file placement. Nothing here proposes a rewrite or a new architecture — every item is a same-file or same-function fix. Every claim below was checked against the current source (`grep/read`, not guessed), and file:line references are given throughout so each item is easy to locate. Findings are grouped to match the questions raised: declaring/building/passing classes, memory hazards, the Model/manager arrangement, dead code, and folder placement — plus a short section on logging/tooling conventions that fell out of the same pass.

Severity legend: **Bug** = produces wrong behaviour today · **Perf/Mem** = wastes CPU or allocates unnecessarily · **Style** = correct but inconsistent with the rest of the codebase · **Dead** = unreachable or unused code.

1. Correctness bugs worth fixing first

These are small, but they're genuine bugs — not style opinions — and each is a one- or two-line fix.

1.1 Field.cf setter crashes with the wrong exception type

src/cosmos/core/field.py:65 — Bug

```
raise TypeError("Unsupported type for Field " + self.name)
```

Field never defines self.name. Passing an unsupported type to the cf setter raises AttributeError: 'Field' object has no attribute 'name' instead of the intended TypeError, so any caller doing except TypeError: ... around a pde.set_params(...) call won't catch it. **Fix:** drop self.name (the class has no name), or reference type(self._coef) in the message.

1.2 CosmosModel.create_ale compares the wrong attribute

src/cosmos/core/model.py:148 — Bug

```
if any(x == y for z in self.ales for x in compartment.domain_id.split('|')
      for y in z.compartment.boundary_id.split('|')):
```

The guard is meant to reject a second ALE field on a domain that already has one. It compares the *new* compartment's domain_id against the *existing* ALE compartments' boundary_id — two different attributes that are usually different strings — so the duplicate-ALE check almost never actually fires. **Fix:** compare domain_id to domain_id on both sides.

1.3 set_config silently accepts typos and never interpolates the key

src/cosmos/config/parameters.py:31–36 — Bug

```
def set_config(**kwargs):
    for key, value in kwargs.items():
        if not hasattr(_current_config, key):
            logger.error("Unknown config parameter: {key}")
            setattr(_current_config, key, value)
```

Two independent problems on the same three lines: the message is missing the f prefix, so it always logs the literal string "Unknown config parameter: {key}" rather than the actual key; and even after detecting an unknown key, the function proceeds to setattr anyway, silently bolting a new attribute onto the shared global Config singleton. A typo like set_config(percision= "float32") doesn't error, doesn't usefully log, and leaves the *real* precision field untouched. **Fix:** make the f string literal and continue/raise on an unknown key instead of setting it anyway.

1.4 Volume ADR's nonlinearity hook breaks for a single-species system

src/cosmos/pde/adr/volume/adr_volume_system_bdf1_model.py:181 vs. the boundary variant's equivalent line — Bug

```
env.update({"u"+str(j+1): self.gfu_old.components[j], "v"+str(j+1): test[j]})
```

For dim == 1, self.gfu is a plain scalar GridFunction with no .components (this is exactly why the class special-cases self.sol = [self.gfu] a few lines above it). add_nonlinearity on a scalar system raises AttributeError at solve time. The boundary variant of the same model (adr_boundary_system_bdf1_model_{stab,nostab}.py) gets this right by using self.sol[j] instead of self.gfu_old.components[j] — the volume variant should match it. (Pinned by tests/unit/pde/adr/test_adr_volume.py::: currently xfail.)

2. Memory and performance

2.1 The stabilised boundary-ADR model rebuilds its whole linear system every time step

`src/cosmos/pde/adr/boundary/adr_boundary_system_bdf1_model_stab.py:128–217` and `:269–271` — Perf/Mem, the most impactful item in this report

Every other time-dependent model in the package (`adr_volume_system_bdf1_model.py`, `adr_boundary_system_bdf1_model_no`, `geometrical_flow_model.py`, `geometrical_flow_stationary_model.py`) follows the same shape: build the symbolic BilinearForm/LinearForm **once**, in `Initialize()`, and on every `Solve()` just update coefficient GridFunctions and call `.Assemble()`. The *stabilised* boundary ADR model is the one outlier — its `Solve()` does this instead:

```
def Solve(self):
    trial, test = self.fes.TnT()
    self.A = BilinearForm(self.fes)      # rebuilt from scratch, every step
    self.F = LinearForm(self.fes)       # rebuilt from scratch, every step
    ...
    dir_bnd_gfu = GridFunction(facet_space) # new allocation, every step
    bnd_gfu = GridFunction(facet_space)    # new allocation, every step
    neu_bnd_gfu = GridFunction(facet_space) # new allocation, every step
    ...
```

and its `PostProcess()` ends with

```
del self.A
del self.inva
del self.F
```

That explicit `del` is a tell: whoever wrote this already noticed the per-step objects were piling up and reached for a band-aid instead of the fix its sibling classes already use. Re-deriving the symbolic form graph every step is measurably slower than reassembling a form built once (NGSolve’s own recommended pattern), and the three GridFunctions allocated per species per step are pure churn. **Fix:** move the `self.fes.TnT()` call, the `self.A` `/` `self.F` = construction, and the `dir_bnd_gfu`/`bnd_gfu`/`neu_bnd_gfu` allocations into `Initialize()` (mirroring the `nostab` file line-for-line), leaving `Solve()` to just update coefficients, `Assemble()`, and solve. Drop the `del`s once the reallocation is gone — they won’t be needed.

2.2 `implicit_solve_step_gauss` copies every PDE’s full vector on every sub-iteration

`src/cosmos/core/step_manager.py:166–196` — Perf/Mem, minor

```
errors_pde[count] = Norm(pde.gfu.vec-old_sol[count])/np.max([...])
old_sol[count] = pde.gfu.vec.Copy()
```

`.Copy()` allocates a brand-new NGSolve vector every sub-iteration, for every PDE, up to `iter_max` (15 by default) times per time step. For an implicit coupling with several PDEs this is a lot of short-lived allocation churn on the hot path. **Fix:** allocate `old_sol[i]` once (outside the `while`), and inside the loop do `old_sol[count].data = pde.gfu.vec` instead of rebinding to a fresh `.Copy()`.

2.3 `_extend_displacement_to_bulk` reassembles the same matrix twice per step

`src/cosmos/core/ale_manager.py:174–196` calling into `:224–231` — Perf, minor

`solve_ale` calls `_extend_displacement_to_bulk` twice per step when both a boundary displacement and a boundary “material” displacement are present (`gfu_bnd_ale` then `gfu_bnd_mat`), and each call does `self.A.Assemble()` from scratch. Since `self.A` only depends on `self.Yo` (fixed for the whole step, updated

once in `finalize`), the second `Assemble()` in the same step recomputes an identical matrix. **Fix:** assemble once per step (e.g. at the top of `solve_ale`) and reuse the factorisation for both extension calls.

2.4 Dead, commented-out code left in place instead of relying on git history

- `src/cosmos/core/ale_manager.py:185–196` — a full commented-out duplicate of the active block immediately above it (the `gfu_bnd_mat/gfu_bnd_ale` extension logic), left as a comment instead of deleted.
- `src/cosmos/utils/generate_meshes.py:248–272` — a ~25-line commented-out spline-based implementation of `generate_boundary_torus`, superseded by the active OCC-based one just below it.

Neither is a runtime cost, but both are readability/style debt — git already preserves the history; delete both blocks.

3. Class declaration, construction, and object design

3.1 Redundant re-assignment right after `super().__init__()`

Style, repeated in five files: `pde/adr/volume/adr_volume_system_bdf1_model.py:29–34`, `pde/adr/boundary/adr_boundary_system_bdf1_model.py:29–34`, `pde/adr/boundary/adr_boundary_system_bdf1_model_nostab.py` (same lines), `pde/distance/volume/distance_volume_model.py:29–34`, `pde/willmore/geometrical_flow_model.py:27–33`, `pde/willmore/geometrical_flow_stationary_model.py:26–32`.

```
super().__init__(name=name, model = model, compartment=compartment)
self.is_bnd = False
self.is_vol = True
self.name = name          # already set by BasePDEModel.__init__
self.model = model        # already set by BasePDEModel.__init__
self.compartment = compartment # already set by BasePDEModel.__init__
```

`BasePDEModel.__init__` (`pde/base.py:26–39`) already assigns `name`, `model`, and `compartment`. Every concrete subclass repeats all three assignments immediately after calling `super().__init__` with the exact same values — harmless, but it's copy-pasted boilerplate across six constructors and invites the two copies to drift (e.g. if a subclass ever normalises `name` before storing it, only one of the two assignments would reflect that). **Fix:** delete the three redundant lines in each subclass; keep only `self.is_bnd/self.is_vol`, which `BasePDEModel` doesn't set to a concrete value on its own.

`is_bnd/is_vol` are themselves static per PDE type (every `ADRVolumeSystemBDF1Model` is `is_vol = True`, always) — they could be class attributes (`is_bnd = False` in the class body) rather than being re-assigned in `__init__` on every instantiation. Small, but it documents the invariant at the class level instead of the instance level.

3.2 `Optional[...]` typing is inconsistent between `base.py` and its subclasses

Style

`pde/base.py:29` correctly types the constructor as `model: Optional[CosmosModel] = None`. Every concrete subclass instead writes `model:CosmosModel = None` (e.g. `adr_volume_system_bdf1_model.py:27`, `distance_volume_model.py:26`, `geometrical_flow_model.py:25`, `geometrical_flow_stationary_model.py:24`, both boundary ADR files) — a type that, read literally, forbids `None` while the default value *is* `None`. Harmless at runtime (Python doesn't enforce it), but it defeats a type checker and is inconsistent with the base class one file away. **Fix:** `Optional[CosmosModel] = None` / `Optional[CosmosCompartment] = None` everywhere `base.py` already got it right.

3.3 Managers are inconsistent about whether they hold a back-reference to `model`

Style, real usability friction

`CosmosCompartment` (`core/compartment.py:19–24`), `CosmosBndALEField` (`core/ale_manager.py:267–271`), and `CosmosVolALEField` (`core/ale_manager.py:429–433`) all store `self.model = model` in their constructor and never need it passed again. `CosmosALEManager` (`core/ale_manager.py:26`), `CosmosStepManager` (`core/step_manager.py:23`), and `CosmosIOManager` (`io/io_manager.py:28`) all *receive* `model` (or, for the latter two, are handed it in `initialize(self, model)`), but never store it — so every one of their methods (`initialize`, `solve_ale`, `finalize`, `solve_step`, `save_step_data`, ...) has to take `model` as an explicit parameter again, even though a `CosmosALEManager` only ever belongs to exactly one `CosmosModel` for its whole lifetime. **Fix:** pick one convention. The natural one here is “managers store `self.model` once, methods stop re-accepting it” — it removes one parameter from ~10 method signatures and matches what the field/compartment classes already do.

3.4 Config kwargs are threaded through four managers with no validation

core/model.py:28–34 — Bug-adjacent, Style

```
self.time = CosmosTimeManager(self.params)
self.step = CosmosStepManager(self.params)
self.ale = CosmosALEManager(self, self.params)
self.io = CosmosIOManager(self.params)
```

CosmosModel.__init__ takes ****kwargs** and hands the *entire, unfiltered* dict to all four managers. Each manager then does its own if 'some_key' in self.params.keys(): ... lookups for just the keys it cares about (e.g. coupling_type/adaptive_timestep in step_manager.py, volume_ALE/surface_ALE/redistribute in ale_manager.py, root/samples/output_callables in io_manager.py, t0/t1/dt in time_manager.py). Nothing ever flags a key that no manager recognises — CosmosModel("m", mesh, t0=0, t1=1, dt=0.1, coupling_type="implicit") (typo) silently falls back to the default 'explicit' coupling with zero feedback. Combined with §1.3, this is the same “silent typo” shape happening in two different places for the same underlying reason: no manager validates its incoming keys against a known set. **Fix (small, local):** at the end of each manager’s initialize(), warn (via logger. warning) on any leftover/unrecognised keys that look like they were meant for that manager but weren’t consumed — or, more simply, have CosmosModel.__init__ maintain one explicit whitelist of accepted top-level keys and raise on anything else, the same way set_params already does elsewhere in the codebase (pde/base.py:57–65, core/model.py:100–107).

4. Dead and unused code

Location	What	Evidence
utils/mesh_fixing.py:246–251 (globally_align_sign) and its helper_pca_third_axis:239–244 utils/generate_meshes.py:349– 355 (__main__ block)	Defined, never called from anywhere in src/ or tests/ References a module-level __all__ that is never defined anywhere in the file, and calls every generator with a geo_only=False kwarg that none of them accept	grep -rn globally_align_sign matches only the def line Running python -m cosmos.utils.generate_meshes raises NameError immediately
src/cosmos/coupling/ (__init__.py, interface.py) 12 files import Draw from ngsolve.webgui/netgen.webgui and never call it: core/step_manager.py, core/ale_manager.py, utils/generate_meshes.py, utils/mesh_fixing.py, utils/dendritic_spine_geom.py (both Draw and DrawGeo), both pde/adr/boundary/*.py, pde/willmore/*.py (both files), pde/adr/volume/adr_volume_system_bdf1_model.py, pde/distance/volume/distance_volume_model.py	Both files are completely empty (0 lines) grep -c "Draw(" is 0 in every one of them	wc -l on both

None of these cost anything at runtime (an unused import is cheap), but `coupling/` in particular reads as a half-started feature — either give it a real `CosmosInterface/coupling` class, or remove the empty subpackage so it stops showing up in the package layout as something that exists. The unused `Draw` imports are worth a single sweep (an IDE “optimize imports” pass handles all 12 in one shot) — right now every file that imports NGSolve’s interactive-notebook `Draw` looks like it has debugging/visualisation code half-wired-in that isn’t actually there.

5. File and folder organisation

5.1 Two unrelated classes share the exact same name

Style, real footgun

pde/adr/boundary/adr_boundary_system_bdf1_model_stab.py:16 and pde/adr/boundary/adr_boundary_system_bdf1_model_nostab.py:16 both define class `ADRBoundarySystemBDF1Model(BasePDEModel)`. They're deliberately interchangeable at the call site (`model.create_pde("s", ADRBoundarySystemBDF1Model, ...)`), which is presumably why they share a name — but it means from `...model_stab import ADRBoundarySystemBDF1Model` and from `...model_nostab import ADRBoundarySystemBDF1Model` are silently different classes with identical `repr/__name__`. A stray wrong import (easy to make — the module names differ only by `_stab/_nostab`) produces no error, just a solver that's quietly missing gradient-jump stabilisation. **Fix:** rename the classes themselves, e.g. `ADRBoundarySystemBDF1StabModel / ADRBoundarySystemBDF1NoStabModel` (or a shared base with `Stab/NoStab` suffixes) — the module filenames already make the distinction, the classes should too.

5.2 Inconsistent nesting depth inside pde/

Style

```
pde/adr/volume/adr_volume_system_bdf1_model.py
pde/adr/boundary/adr_boundary_system_bdf1_model_{stab,nostab}.py
pde/distance/volume/distance_volume_model.py
pde/willmore/geometrical_flow_model.py          <- no volume/boundary split
pde/willmore/geometrical_flow_stationary_model.py <- no volume/boundary split
```

`adr/` and `distance/` both nest one level deeper (`volume/`, `boundary/`) even though, today, each of those leaf folders holds exactly one file. `willmore/` skips that level entirely, even though both `willmore` models are boundary-only — which is fine, but it means the same organising principle (“split by volume vs. boundary”) isn't applied consistently across the three PDE families. **Fix (pick one, either is reasonable, just be consistent):** either drop the `volume/`/`boundary/` subfolders for `adr/` and `distance/` (they add a directory for a single file) and disambiguate via filename instead (`adr_volume_...py`, `adr_boundary_...py`, already true today), or add a `boundary/` subfolder under `willmore/` to match `adr/`'s convention.

5.3 utils/dendritic_spine_geom.py is domain-specific, not a generic utility

Style

`utils/tools.py` (generic FEM helper), `utils/generate_meshes.py` (generic geometry factory), and `utils/mesh_fixing.py` (generic `.vol` file editing) are all reusable across any Cosmos application. `utils/dendritic_spine_geom.py` is not — it's one specific biological geometry (a synaptic spine mesh) baked into the same package as the generic tools. This is also *why* it's duplicated: `tests/pde_bio/physical/Bonilla-QuintanaRangamani2024/dendritic_spine_geom.py` is a second, independent copy of essentially the same geometry code, kept inside the test tree instead of importing the one in `src/`. **Fix:** move `utils/dendritic_spine_geom.py` to an application/example location (e.g. a `cosmos.examples.bio` or `cosmos.geometries.bio` module, or simply `utils/bio/dendritic_spine_geom.py` if it should stay in the package), and point the `tests/pde_bio` copy at the real one instead of vendoring its own.

5.4 Subpackage `__init__.py` files are all empty

Style

Every `__init__.py` under `core/`, `pde/`, `io/`, `config/`, `utils/`, `coupling/` is empty — only the top-level `cosmos/__init__.py` re-exports anything (`Config`, `get_config`, `set_config`). Every PDE model, mesh generator, and manager has to be imported via its full module path (from `cosmos.pde.adr.boundary.adr_boundary_system_bdf1_model_nostab import ADRBoundarySystemBDF1Model`), which is exactly the pattern used throughout `docs/index.md`'s own

quick-start. This is workable, but it's also plausibly *why* the old (now-stale) test suite expected from `cosmos import *` to expose solver classes directly — that ergonomic surface used to exist (or was planned) and no longer does. **Fix:** re-export the concrete, stable classes at the natural subpackage level — e.g. `cosmos/pde/__init__.py` exposing `ADRVolumeSystemBDF1Model`, both boundary variants (once renamed per §5.1), `DistanceVolumeModel`, `GeometricalFlowModel`, `GeometricalFlowStationaryModel` — so callers can write from `cosmos.pde import ADRVVolumeSystemBDF1Model` instead of the four-level path. Doesn't require moving any files, just adding `__all__`/imports to the existing empty `__init__.pys`.

6. Logging, error handling, and tooling conventions

6.1 A library shouldn't call `logging.basicConfig()` on import

`src/cosmos/io/logger.py:4-7`, executed unconditionally via `cosmos/__init__.py:5` (from `cosmos.io.logger import *`) — Style, established Python practice

```
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(name)s: %(message)s'
)
```

This runs the moment anyone does `import cosmos`. Per the standard library's own logging HOWTO, only the *application* (the end script) should call `basicConfig` / attach handlers to the root logger — a library is supposed to only do `logger = logging.getLogger(__name__)` and let the importing application decide how (or whether) to configure output. As written, any downstream project that imports `cosmos` gets its root logger silently reconfigured (level, format, handlers) the moment the import happens, which can clobber that project's own logging setup if `cosmos` is imported first. **Fix:** delete the `basicConfig` call (or gate it behind an explicit `cosmos.enable_default_logging()` opt-in function); each module already does the correct `logger = logging.getLogger(__name__)` on its own.

6.2 `print()` instead of the already-imported logger, inconsistently

Style

Every module sets up `logger = logging.getLogger(__name__)` at the top (except `utils/mesh_fixing.py`, which has no logging import at all — see below), but plain `print()` is still used for status/progress output in several places:

- `core/step_manager.py` — 6 `print(...)` calls (e.g. :107, :114, :142, :201, :203) vs. only 2 `logger.debug(...)` calls in the same file, for what is clearly the same category of message (per-iteration solver diagnostics).
- `io/io_manager.py` — 22 `print(...)` calls (e.g. the 'PRINTED!' at :119) and **zero** uses of the logger it defines at the top of the file.
- `utils/mesh_fixing.py` — 19 `print(...)` calls, including inside an `except Exception as e:` block at :114-116 — and this file never imports logging at all, so there isn't even a logger object available to switch to without adding the import first.

Fix: route these through `logger.info/logger.debug/logger.warning` as appropriate. This isn't just style — it's what makes §6.1 worth fixing: once callers can configure log levels/handlers, `print()` calls bypass that configuration entirely (they can't be silenced, redirected, or leveled).

6.3 Swallowed exception loses its original type and traceback

`src/cosmos/utils/mesh_fixing.py:107-116` — Style

```
try:
    ...
    self.mesh = Mesh(self.filename)
    ...
except Exception as e:
    print('Error:', e)
    raise OSError('Impossible to generate NgSolve mesh from given filename')
```

Catching a broad `Exception`, printing it, and re-raising a *different* exception type without `raise ... from e` discards the original traceback — whatever actually went wrong (bad path, corrupt `.vol` syntax, permissions) is reduced to one printed line that's easy to miss and impossible to except `FileNotFoundError`

around. **Fix:** either let the original exception propagate (drop the try/except entirely — `Mesh(...)` already raises something meaningful), or keep it but use `raise OSError(...) from e` and `logger.exception(...)` instead of `print`.

6.4 One stray formatting slip

`src/cosmos/utils/tools.py:2` — cosmetic

```
from ngsolve .solvers import *
```

Extra space before `.solvers` — harmless to the interpreter, but it's a sign nothing in this project runs through an autoformatter. See §6.5.

6.5 No lint/format/type-check tooling configured

Style, established practice

`pyproject.toml` has no `[tool.ruff]`, `[tool.black]`, or `[tool.mypy]` section, and there's no `.github/workflows` (no CI at all). Given how many of the items above are exactly what `ruff` (unused imports, unused variables, f-string-without-placeholder typos like §1.3) or `mypy` (`Optional[...]` mismatches like §3.2) would catch automatically and for free, adding a minimal `ruff check + ruff format` pass — even just locally, without CI — would have caught roughly a third of this report mechanically. A `py.typed` marker file (PEP 561, currently absent from `src/cosmos/`) is a one-line addition once the `Optional[...]` annotations are consistent, and lets downstream users type-check against `cosmos`.

7. Summary table

Ordered roughly by impact-to-effort ratio.

#	Item	File(s)	Type	Effort
2.1	Boundary-ADR stab model rebuilds A/F/GridFunctions every step	adr_boundary_system_Perf/Model_stab.py	Perf/Model	Medium (move code between two methods)
1.4	Volume ADR nonlinearity breaks for dim=1	adr_volume_system_bdf1model.py:181	Bug	Trivial
1.2	create_ale duplicate-domain check compares wrong attribute	core/model.py:148	Bug	Trivial
1.3	set_config missing f-string + sets unknown keys anyway	config/parameters.py:36	Bug	Trivial
1.1	Field.cf setter references undefined self.name	core/field.py:65	Bug	Trivial
6.1	Library calls log- ging.basicConfig() on import	io/logger.py, cos- mos/__init__.py	Style	Trivial
5.1	Two classes named ADRBoundarySys- temBDF1Model in different modules	adr_boundary_system_style_model_{stab, no- stab}.py	Style	Small (rename + update call sites)
3.4 / 1.3	No validation of **kwargs passed into managers/config	core/model.py, con- fig/parameters.py	Style	Small
2.2	.Copy() per sub-iteration in implicit Gauss-Seidel loop	core/step_manager.py:196	Perf	Small
2.3	Redundant matrix reassembly in ALE bulk extension	core/ale_manager.py:231	Perf	Small
3.3	Managers inconsistently store (or don't store) self.model	core/ale_manager.py, Style core/step_manager.py, io/io_manager.py	Style	Small, mechanical
3.1	Redundant self.name/self.model/self.compartment after super().__init__	6 PDE model files	Style	Trivial, mechanical

#	Item	File(s)	Type	Effort
3.2	model:CosmosModel = None instead of Optional[CosmosModel] = None	6 PDE model files	Style	Trivial, mechanical
4	Unused Draw imports (12 files), dead globally_align_sign, broken generate_meshes.py __main__	see §4	Dead	Trivial
2.4	Commented-out dead code blocks	ale_manager.py, generate_meshes.py	Dead	Trivial
6.2	print() instead of logger.*	step_manager.py, io_manager.py, mesh_fixing.py	Style	Small
6.3	Broad except Exception swallows original error	mesh_fixing.py:107-116	Style	Trivial
5.3	dendritic_spine_geometry (bio-specific) lives in generic utils/, and is duplicated in tests/	utils/dendritic_spine_geometry.py	Organisation	Small
5.4	Empty subpackage __init__.py force deep import paths	all of src/cosmos/*/__init__.py	Organisation	Small
4	Empty coupling/ subpackage	coupling/__init__.py	Dead	Decision needed (implement or remove)
5.2	Inconsistent volume//boundary/ nesting across pde/adr, pde/distance, pde/willmore	coupling/interface.py pde/	Organisation	Small, mostly file moves
6.4 / 6.5	No formatter/linter/CI, one stray-space import	pyproject.toml, tools.py:2	Tooling	Small setup, ongoing payoff

None of these require touching the numerical/variational-form logic itself (with the partial exception of §2.1, which only moves *where* the forms are built, not what they compute) — every item is a local, mechanical change that a single pass through the listed files can address without risking the physics.